

CRUD Operations in Entity Framework Core (EF Core)

In this article, I will discuss **CRUD Operations in Entity Framework Core (EF Core)**. Please read our previous article discussing [Database Connection String in Entity Framework Core](#). We will work with the same example we have worked on so far.

What are CRUD Operations?

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations that can be performed on data in a database:

- **Create:** Inserting new records into a database.
- **Read:** Retrieving data from the database, often referred to as querying.
- **Update:** Modifying existing records in the database.
- **Delete:** Removing records from the database.

How Do We Perform CRUD Operations in Entity Framework Core (EF Core)?

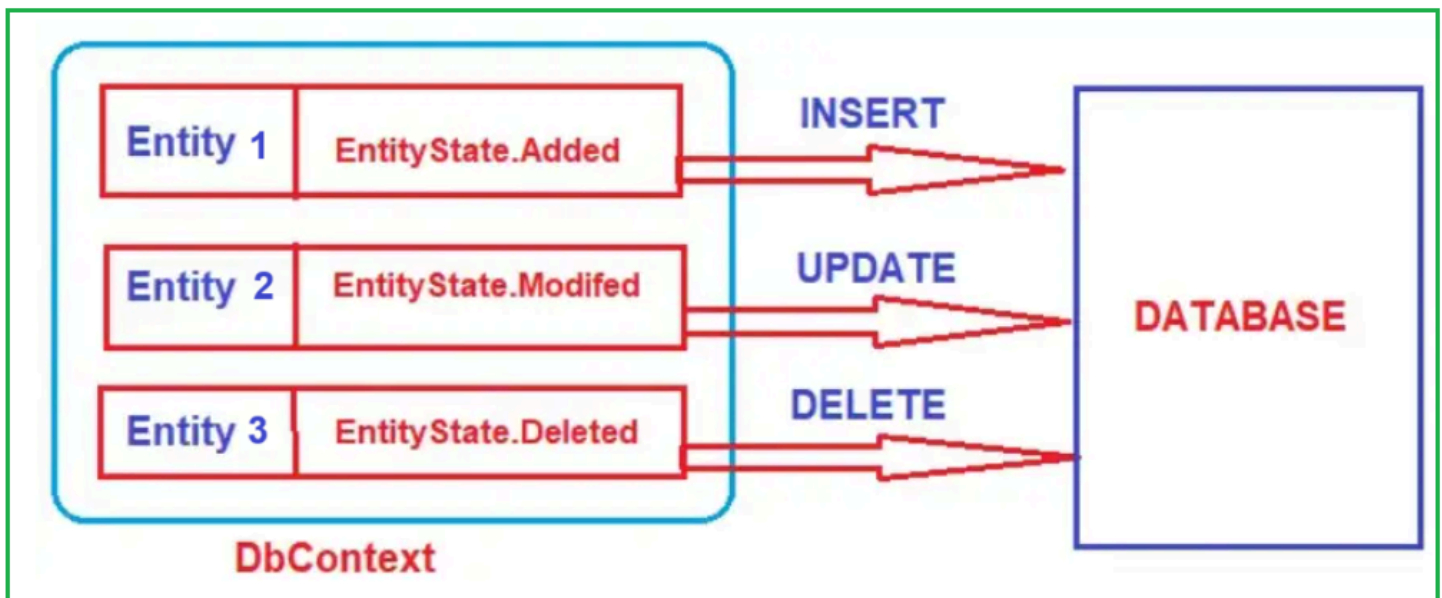
To perform Insert, Update, and Delete operations in Entity Framework Core (EF Core), there are two persistence scenarios: Connected and Disconnected.

- **Connected Scenario:** In this scenario, the same DbContext instance is used to retrieve and save the entities. This means that the DbContext tracks the entities' states throughout their lifecycle.
- **Disconnected Scenario:** In this scenario, Different DbContext instances are used to retrieve and save the entities. For example, one DbContext instance might retrieve the data, and another DbContext instance saves the changes to the database. Since the DbContext instances are different, the state of the entities is not automatically tracked across them.

In this article, we will focus on performing CRUD operations in the connected scenario. In our upcoming articles, we will discuss how to perform CRUD operations in a disconnected scenario.

EF Core CRUD Operation in Connected Scenario:

In the Connected Scenario, the DbContext tracks Entities, and their state (e.g., Added, Modified, Deleted) determines whether they will be inserted, updated, or deleted in the database. When the SaveChanges() method is called on the DbContext instance, EF Core automatically generates and executes the corresponding SQL INSERT, UPDATE, or DELETE statements based on the entity's state. The following diagram shows the connected scenario's CUD (Create, Update, Delete) operations.



The above diagram shows how Entity Framework Core (EF Core) manages the states of entities within a DbContext and how these states determine the CRUD (Create, Read, Update, Delete) operations performed on the database.

Entities and Their States:

The above diagram shows three entities, each with a different state:

- Entity 1: Its state is EntityState.Added.
- Entity 2: Its state is EntityState.Modified.
- Entity 3: Its state is EntityState.Deleted.

DbContext:

A single instance of DbContext tracks all these entities and is responsible for managing them and their states. The DbContext tracks changes to the entities and determines what SQL operations (INSERT, UPDATE, DELETE) need to be performed on the database when SaveChanges() is called.

CRUD Operations:

Based on the state of each entity, the DbContext will generate the corresponding SQL commands:

- **Entity with EntityState.Added:** An INSERT command is generated, meaning this entity will be added to the database.
- **Entity with EntityState.Modified:** An UPDATE command is generated, meaning the existing record in the database will be updated with the changes made to this entity.
- **Entity with EntityState.Deleted:** A DELETE command is generated, meaning this entity will be removed from the database.

Interaction with the Database:

When the SaveChanges() method is invoked, EF Core executes the appropriate SQL statements (INSERT, UPDATE, DELETE) based on the entity states, reflecting the changes in the actual database.

What is EF Core Logging?

EF Core Logging refers to the process of capturing and recording the internal operations of Entity Framework Core, such as the SQL queries it generates, the query execution times, and other relevant activities that occur during runtime. Logging is crucial for developers as it provides insights into what EF Core is doing behind the scenes, including the SQL queries generated, how they are executed, and their interaction with the database.

Why Do We Need to Enable EF Core Logging?

Enabling EF Core Logging is beneficial for several reasons. They are as follows:

- **Debugging and Troubleshooting:** By understanding the exact SQL commands that EF Core generates, developers can debug issues related to data manipulation and retrieval more effectively.
- **Performance Optimization:** Reviewing the queries generated by EF Core allows developers to identify performance bottlenecks, such as inefficient queries, unnecessary data loading (e.g., N+1 query problems), or slow database operations. By optimizing these queries, adding indexes, and making other improvements, application performance can be enhanced.
- **Monitoring:** Logging is also useful for monitoring and auditing database operations, especially in production environments where it's important to track the application's behavior over time.
- **Understanding EF Core Behavior:** For developers new to EF Core, logging provides valuable insights into how EF Core translates LINQ queries into SQL and how it manages entity states, relationships, and transactions. This understanding can accelerate the learning process and improve coding practices.

How Do We Enable EF Core Logging?

EF Core logging can be enabled and configured in various ways, depending on the type of application (Console, ASP.NET Core, etc.) and the logging framework you are using. In a Console application, EF Core logging can be enabled to output logs directly to the Console window. This approach requires minimal configuration and does not require additional NuGet packages.

To enable logging in EF Core, you can use the `LogTo` method on the `DbContextOptionsBuilder` instance within the `OnConfiguring` method. This method allows you to specify where the logs should be displayed (in this case, using `Console.WriteLine`) and the level of logs you want to capture (e.g., Information, Trace, Debug, Error, Warning). So, for a better understanding please modify the `EFCoreDbContext` class as shown below to display the EF Core Log Information on the Console window.

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    // EFCoreDbContext is your custom DbContext class that extends the base DbContext by EF Core.
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Display the generated SQL queries in the Console window
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            // Configure the connection string
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB1;Trusted_Connection=True;TrustServerCertificate=True");
            // DbSet<Student> corresponds to the Students table in the database.
            // It allows EF Core to track and manage Student entities.
            public DbSet<Student> Students { get; set; }
        }
    }
}
```

```

        // DbSet<Branch> corresponds to the Branches table in the database.
        // It allows EF Core to track and manage Branch entities.
        public DbSet<Branch> Branches { get; set; }
    }
}

```

CRUD Operations using EF Core:

Let us proceed and understand How to Perform CRUD Operations using EF Core. Before proceeding, first, delete all the data from the database tables by executing the following commands:

```

USE EFCoreDB1
GO
TRUNCATE TABLE Students;
DELETE FROM Branches;

```

Create Operation using Entity Framework Core:

Create Operation means we need to add a new object. Adding a new object using Entity Framework Core is very simple. First, you need to create an instance of the Entity we want to add to the database. Once you have created the instance, we need to call the **Add Method** of the **DbSet** or **DbContext** class and pass the entity. Finally, call the **SaveChanges** method on the DbContext object, which will insert the new record into the database by generating the INSERT SQL Statement.

Differences Between DbSet.Add and DbContext.Add Methods of EF Core:

In Entity Framework Core (EF Core), both DbSet.Add and DbContext.Add methods are used to add entities to the context so that they can be inserted into the database when SaveChanges is called. However, there are some little differences between them.

- **DbSet.Add Method:** The DbSet.Add method is used to add a single entity of a specific type to the DbSet. When we call DbSet.Add, the entity is added to the context in the Added state, which means EF Core will insert this entity into the database when SaveChanges is called.
- **DbContext.Add Method:** The DbContext.Add method is a more generic method available on the DbContext. It can be used to add entities of any type to the context. When we call DbContext.Add, it can add an entity to any of the DbSets that are part of the context. It determines the appropriate DbSet based on the type of the entity. This method is useful when working with different types of entities, and we don't want to specify the DbSet explicitly.

Example to Perform Create Database Operation using EF Core:

For a better understanding, modify the Program class as follows. In the example below, I show the use of both DbSet.Add and DbContext.Add methods. The following code is self-explained, so please read the comment lines for a better understanding.

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try

```

```

{
    // Create an instance of your EFCoreDbContext to interact
with the database
    using (var context = new EFCoreDbContext())
    {
        // Create two new Branch objects
        var CSEBranch = new Branch
        {
            BranchName = "Computer Science",
            Description = "Focuses on software development and
computing technologies.",
            PhoneNumber = "123-456-7890",
            Email = "cs@dotnettutorials.net"
        };
        var ElectricalBranch = new Branch
        {
            BranchName = "Electrical Engineering",
            Description = "Focuses on electrical systems and
circuit design.",
            PhoneNumber = "987-654-3210",
            Email = "ee@dotnettutorials.net"
        };
        // Adding the Branch objects to the Branches DbSet
        // This prepares the object to be inserted into the
database
        context.Branches.Add(CSEBranch);
        context.Branches.Add(ElectricalBranch);
        // Alternatively, you can use the DbContext.Add method to
add entities
        // context.Add(CSEBranch);
        // context.Add(ElectricalBranch);
        // Create two new Student objects
        var student1 = new Student
        {
            FirstName = "Pranaya",
            LastName = "Rout",
            DateOfBirth = new DateTime(2000, 1, 15),
            Gender = "Female",
            Email = "Pranaya.Rout@dotnettutorials.net",
            PhoneNumber = "555-1234",
            EnrollmentDate = DateTime.Now,
            Branch = CSEBranch // Assign the Computer Science
branch
        };
        var student2 = new Student
        {
            FirstName = "Rakesh",
            LastName = "Kumar",
            DateOfBirth = new DateTime(1999, 10, 22),
            Gender = "Male",
            Email = "Rakesh.Kumar@dotnettutorials.net",

```


SELECT * FROM Students;
SELECT * FROM Branches;

Results

Messages

StudentId	FirstName	LastName	DateOfBirth	Gender	Email	PhoneNumber	EnrollmentDate	BranchId
1	Pranaya	Rout	2000-01-15 ...	Female	Pranaya.Rout...	555-1234	2024-09-01 ...	3
2	Rakesh	Kumar	1999-10-22 ...	Male	Rakesh.Kumar...	555-5678	2024-09-01 ...	4

BranchId	BranchName	Description	PhoneNumber	Email
3	Computer Science	Focuses on software development and computing te...	123-456-7890	cs@dotnettutori...
4	Electrical Engineering	Focuses on electrical systems and circuit design.	987-654-3210	ee@dotnettutor...

What are Update Operations?

In Entity Framework Core, update operations refer to modifying existing data in the database. When we perform an update operation, we typically:

- Retrieve an entity from the database.
- Modify the properties of the entity that we want to update.
- Save the changes back to the database.

When Do We Need to Perform Update Operations?

Update operations are needed when we want to change existing data in our database. This can include:

- Correcting errors in the data.
- Updating records to reflect new information (e.g., changing a student’s contact details).
- Modifying multiple properties of an entity based on new requirements or user inputs.

How Do We Perform Update Operations using EF Core?

As discussed in the Connected Environment, Entity Framework Core keeps track of all the entities retrieved using the context object. Therefore, when we modify any entity data, the Entity Framework Core will automatically mark the Entity State as Modified. When the SaveChanges method is called, it updates the updated data into the underlying database by generating the UPDATE SQL Statement.

For a better understanding, please modify the Program class as follows. Here, we are updating the Student whose ID is 1. The following example code is self-explained, so please go through the comment lines.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Create an instance of your EFCoreDbContext to interact
                with the database
            }
        }
    }
}
```

```

using (var context = new EFCoreDbContext())
{
    // Retrieve the student whose Id is 1 from the database
using Find method
    // The Find method takes the Primary Key
    var student = context.Students.Find(1);

    // Check if a student was found with ID 1
    if (student != null)
    {
        // Display the original data before updating
        Console.WriteLine("Original Student Data:");
        Console.WriteLine($"Name: {student.FirstName}
{student.LastName}");
        Console.WriteLine($"Email: {student.Email}");
        // Modify the student's properties that need to be
updated
name
name
        student.FirstName = "Paresh"; // Changing the first
name
        student.LastName = "Mohanty"; // Changing the last
name
        student.Email = "Paresh.Mohanty@dotnettutritals.net";
// Changing the email
        // Save the changes to the database
        context.SaveChanges();
        // Display a success message on the console
        Console.WriteLine("Student data has been successfully
updated.");
        // Display the updated data
        Console.WriteLine("Updated Student Data:");
        Console.WriteLine($"Name: {student.FirstName}
{student.LastName}");
        Console.WriteLine($"Email: {student.Email}");
    }
    else
    {
        // Display a message if no student was found
        Console.WriteLine("No student found to update.");
    }
}
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}"); ;
}
}
}
}

```

In the above example, first, we retrieve the student from the database using the **DbSet Find** method, which generates a SELECT SQL query. When we modify the FirstName, LastName, and Email, the context object sets its Entity State to

Modified. When we call the `SaveChanges()` method, it builds and executes the Update SQL statement in the database. Run the above code, and you will get the following output.

```
info: 01-09-2024 20:58:20.808 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (32ms) [Parameters=[@__p_0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[Email], [s].[EnrollmentDate], [s].[FirstNam
e], [s].[Gender], [s].[LastName], [s].[PhoneNumber]
      FROM [Students] AS [s]
      WHERE [s].[StudentId] = @__p_0
Original Student Data:
Name: Pranaya Rout
Email: Pranaya.Rout@dotnettutorials.net
info: 01-09-2024 20:58:20.885 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (9ms) [Parameters=[@p3=?' (DbType = Int32), @p0=?' (Size = 4000), @p1=?' (Size = 4000), @p2=
'? (Size = 4000)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      UPDATE [Students] SET [Email] = @p0, [FirstName] = @p1, [LastName] = @p2
      OUTPUT 1
      WHERE [StudentId] = @p3;
Student data has been successfully updated.
Updated Student Data:
Name: Paresh Mohanty
Email: Paresh.Mohanty@dotnettutorials.net
```

Note: The context object in Entity Framework Core keeps track of the columns of an entity that are modified, and based on the modified columns, it will generate the UPDATE SQL statement. Here, the primary key column will be used in the where condition. If one column value is not updated, it will not include that column while updating the database. So you can see the UPDATE statement. It only updates the First Name, Last Name, and Email columns.

What Are Delete Operations?

Delete operations in Entity Framework Core involve removing an entity from the database. When we perform a delete operation, the entity and its associated data are permanently removed from the database. Delete operations are needed when:

- We want to remove records that are no longer needed.
- We need to clean up data that is obsolete or incorrect.
- We are implementing business logic that requires removing specific data from the system.

How Do We Perform Delete Operation in Entity Framework Core?

To delete an entity using Entity Framework Core, we need to use the `Remove` method of the `DbSet` or `DbContext` object. The `DbSet` or `DbContext` `Remove` method works for existing and newly added entities tracked by the context object.

Calling the `Remove` method on an existing entity being tracked by the context object will not immediately delete the entity from the database. Instead, it will mark the entity state as `Deleted`. When we call the `SaveChanges` method, the Entity will be deleted from the database by generating and executing the `DELETE` SQL Statement. Once the `SaveChanges` method is executed successfully, it will mark the entity state as `Detached`, which means the context object will not track the entity.

In the following example, we remove a student whose `StudentId` 1 from the database. The following example code is self-explained, so please go through the comment lines.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
```

```

{
    try
    {
        // Create an instance of your EFCoreDbContext to interact
with the database
        using (var context = new EFCoreDbContext())
        {
            // Retrieve the student with ID 1 from the database using
LINQ's FirstOrDefault method
            var student = context.Students.FirstOrDefault(s =>
s.StudentId == 1);
            // Check if a student with ID 1 was found to avoid null
reference exceptions
            if (student != null)
            {
                // Display the student data before deletion
                Console.WriteLine("Student Data Before Deletion:");
                Console.WriteLine($"ID: {student.StudentId}");
                Console.WriteLine($"Name: {student.FirstName}
{student.LastName}");
                Console.WriteLine($"Email: {student.Email}");
                // Remove the student entity from the DbSet
                // This marks the entity for deletion in the context
                context.Students.Remove(student);
                // Alternatively, you can use the DbContext.Remove
method to remove entities
                // context.Remove(student);
                // Save the changes to the database
                // This actually performs the DELETE operation in the
database
                context.SaveChanges();
                // Display a success message on the console
                Console.WriteLine("Student record has been
successfully deleted from the database.");
            }
            else
            {
                // Display a message if no student with the specified
ID was found
                Console.WriteLine("No student found with ID 1 to
delete.");
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}"); ;
    }
}
}

```

In the above example, **context.Students.Remove(student)**; mark the **student** entity state as Deleted. When we call the **SaveChanges** method on the Context object, the Entity Framework Core generates and executes the DELETE SQL Statement in the database. Run the above application code, and you should get the following output.

```
info: 01-09-2024 21:07:18.188 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
       Executed DbCommand (19ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
       SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[Email], [s].[EnrollmentDate], [s].[FirstNam
e], [s].[Gender], [s].[LastName], [s].[PhoneNumber]
       FROM [Students] AS [s]
       WHERE [s].[StudentId] = 1
Student Data Before Deletion:
ID: 1
Name: Paresh Mohanty
Email: Paresh.Mohanty@dotnettutorials.net
info: 01-09-2024 21:07:18.290 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
       Executed DbCommand (22ms) [Parameters=[@p0='?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
       SET IMPLICIT_TRANSACTIONS OFF;
       SET NOCOUNT ON;
       DELETE FROM [Students]
       OUTPUT 1
       WHERE [StudentId] = @p0;
Student record has been successfully deleted from the database.
```

In the next article, I will discuss [Entity States in Entity Framework Core](#). In this article, I explain How to Perform **CRUD Operations in Entity Framework Core (EF Core)**. I hope you enjoy this CRUD Operations in EF Core article.